EVALUATION OF RANDOM TESTING IN A BANKING APPLICATION

Name: Kaviya V	Reg. No.: 192421022
Subject: Software Testing	Topic: Black Box Testing

1. Introduction

Black box testing evaluates software functionality by providing inputs and checking outputs against specified requirements, without examining the internal code. In the given scenario, the QA engineer relies on random testing and claims high coverage, but defects in transaction limits and boundary conditions remain undetected. This indicates that the test strategy is not sufficiently systematic or risk-focused.

2. Errors in the Testing Strategy and Coverage Assumptions

- Unstructured test selection: Random inputs may repeatedly exercise ordinary transactions while rarely reaching important edge cases or unusual combinations.
- Unsupported coverage claim: A large number of random tests does not automatically mean that requirements, input classes, decision outcomes, or boundary conditions have been covered.
- Failure to test transaction limits: The strategy may omit minimum, maximum, just-below-limit, and just-above-limit transaction values.
- Lack of requirement traceability: Tests are not clearly mapped to banking rules such as daily withdrawal limits, transfer caps, account balances, and permitted transaction amounts.
- Insufficient negative testing: Invalid amounts, zero or negative values, malformed inputs, insufficient balances, and exceeded limits may not be deliberately tested.
- Weak reproducibility: If random inputs and seeds are not recorded, a failure may be difficult to reproduce and verify after a fix.
- No risk-based prioritization: High-impact functions such as money transfer, withdrawal, and transaction-limit enforcement are not given the focused testing they require.

3. Root Causes of Missing Critical Defects

- Boundary-value omissions: Defects often occur at the edges of valid ranges, especially when comparisons use incorrect operators such as<instead of<=.
- Incomplete equivalence classes: Valid and invalid input groups are not identified, so some important categories receive little or no testing.
- Missing business-rule analysis: The test design does not translate banking requirements into explicit expected results.

- Randomness without constraints: Random values are not guaranteed to include critical limits, invalid values, or rare combinations.
- Inadequate coverage measurement: The engineer measures test quantity or execution count instead of requirement coverage and risk-relevant scenario coverage.
- Insufficient negative and state-based scenarios: The approach may not verify how balances, daily totals, failed transactions, and repeated requests behave as transaction state changes.

4. Appropriate Black Box Testing Techniques

1. Equivalence Partitioning: Divide inputs into valid and invalid groups. For example, if a transaction amount must be between ₹ 1 and ₹ 50,000 inclusive, test representative values from the valid range and invalid values such as ₹ 0, a negative amount, and values above ₹ 50,000.
2. Boundary Value Analysis: Test values at and around each limit. For a permitted amount of ₹ 1 to ₹ 50,000, use ₹ 0, ₹ 1, ₹ 2, ₹ 49,999, ₹ 50,000, and ₹ 50,001. Apply the same principle to daily limits and account-balance thresholds.
3. Decision Table Testing: Create a table of conditions and expected actions. Conditions may include sufficient balance, valid account status, transaction amount within the limit, and daily limit not exceeded. Verify that each meaningful condition combination produces the correct approval or rejection.
4. State Transition Testing: Model account and transaction states, such as initiated, authorized, rejected, completed, and reversed. Test valid and invalid transitions, including failures, retries, and reversals.
5. Use Case Testing: Test end-to-end scenarios such as a normal transfer, a transfer exceeding the daily limit, insufficient funds, repeated submission, and a failed transaction followed by a retry.
6. Negative Testing and Error Guessing: Deliberately supply invalid amounts, missing fields, incorrect formats, unauthorized requests, and repeated requests to check validation and safe error handling.
7. Pairwise or Combinatorial Testing: Where many inputs interact, systematically cover pairs or selected combinations such as account type, transaction channel, transaction amount category, and account status. Use risk-based testing for combinations with high financial impact.

5. Recommended Corrected Testing Process

8. Review the banking requirements and identify all limits, validation rules, transaction states, and high-risk operations.

9. Create a requirement-to-test traceability matrix so each business rule has one or more explicit test cases.
10. Define equivalence classes and boundary values for every transaction amount, balance, daily cap, and relevant input field.
11. Design decision tables for approval and rejection rules, including interactions between balance, account status, and limits.
12. Add state-transition and end-to-end use cases for success, failure, retry, reversal, and duplicate submission.
13. Retain random testing as a supplementary technique. Record the random seed and inputs so failures can be reproduced, and ensure random tests do not replace structured cases.
14. Measure adequacy using requirement coverage, boundary coverage, decision-rule coverage, and relevant state-transition coverage. Re-test defects after fixes and run regression tests.

6. How These Techniques Improve Test Adequacy and Reliability

- Boundary Value Analysis directly targets off-by-one defects and incorrect handling at minimum and maximum transaction limits.
- Equivalence Partitioning reduces redundant tests while ensuring representative valid and invalid input classes are checked.
- Decision tables verify combinations of business conditions and help ensure that transaction approval rules are consistently enforced.
- State Transition Testing checks that transactions move only through permitted states and that failures, retries, and reversals do not corrupt account state.
- Use cases validate realistic end-to-end workflows and confirm that individual rules work correctly together.
- Traceability and explicit coverage criteria make the coverage claim measurable and auditable rather than based on the number of tests executed.
- Reproducible random tests can still discover unexpected defects, while structured tests provide repeatable coverage of known risks.

7. Conclusion

Random testing alone is inadequate for a banking application because it does not guarantee that critical requirements, transaction limits, or boundary conditions are exercised. The main errors are unstructured input selection, unsupported coverage assumptions, missing negative tests, and weak requirement traceability. A stronger black box strategy combines equivalence partitioning, boundary value analysis, decision tables, state transition testing, and end-to-end use cases. Random testing can remain as a supplement. Together, these techniques provide

more systematic, measurable, and repeatable evidence of test adequacy and improve confidence in the reliability of financial transactions.